

TERRAFORM TASK-01

Title

Implementation of Terraform Infrastructure Automation Using GitHub and Jenkins CI/CD Pipeline

Objective

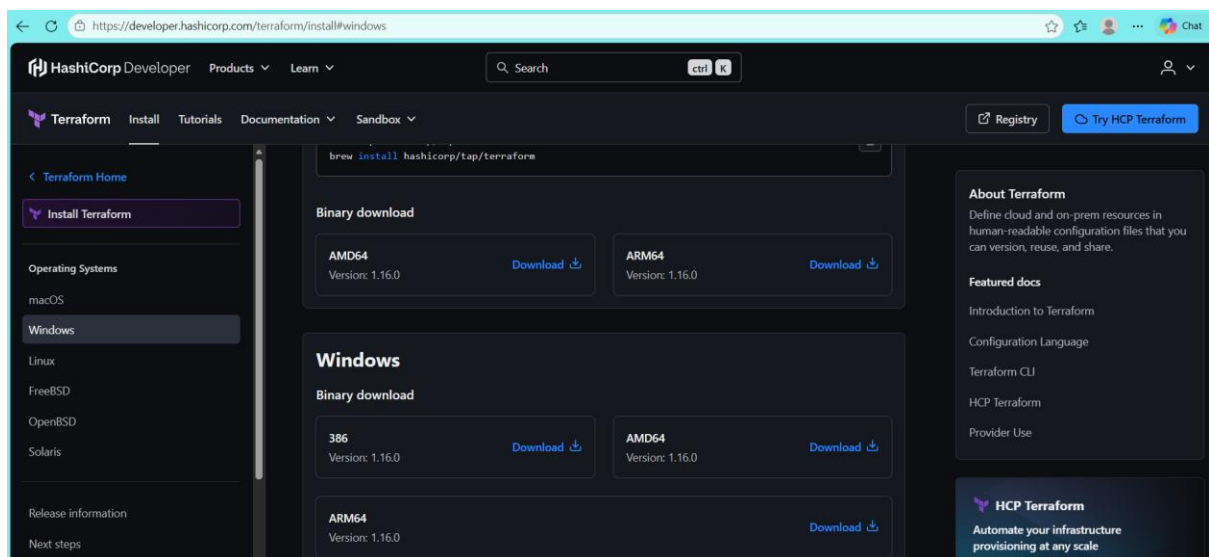
The objective of this task is to install and configure Terraform, write basic Terraform configurations, manage infrastructure state, and automate Terraform execution using a Jenkins CI/CD pipeline integrated with a GitHub repository. The goal is to demonstrate Infrastructure as Code (IaC) practices along with continuous integration and automated deployment using Jenkins.

Prerequisites

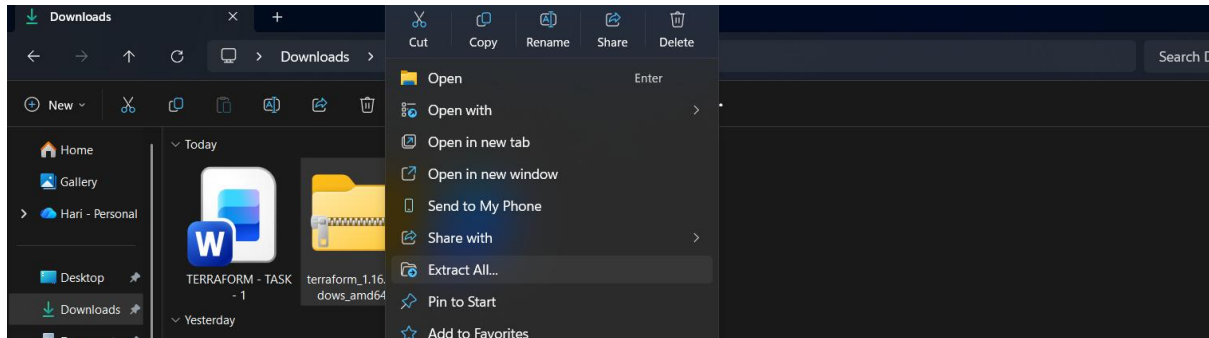
- Windows system with internet access
- Visual Studio Code installed
- Terraform installed on local machine
- GitHub account
- Jenkins server running on Linux (EC2 instance)
- Git installed on Jenkins server
- Terraform installed on Jenkins server

1) Install Terraform on your PC

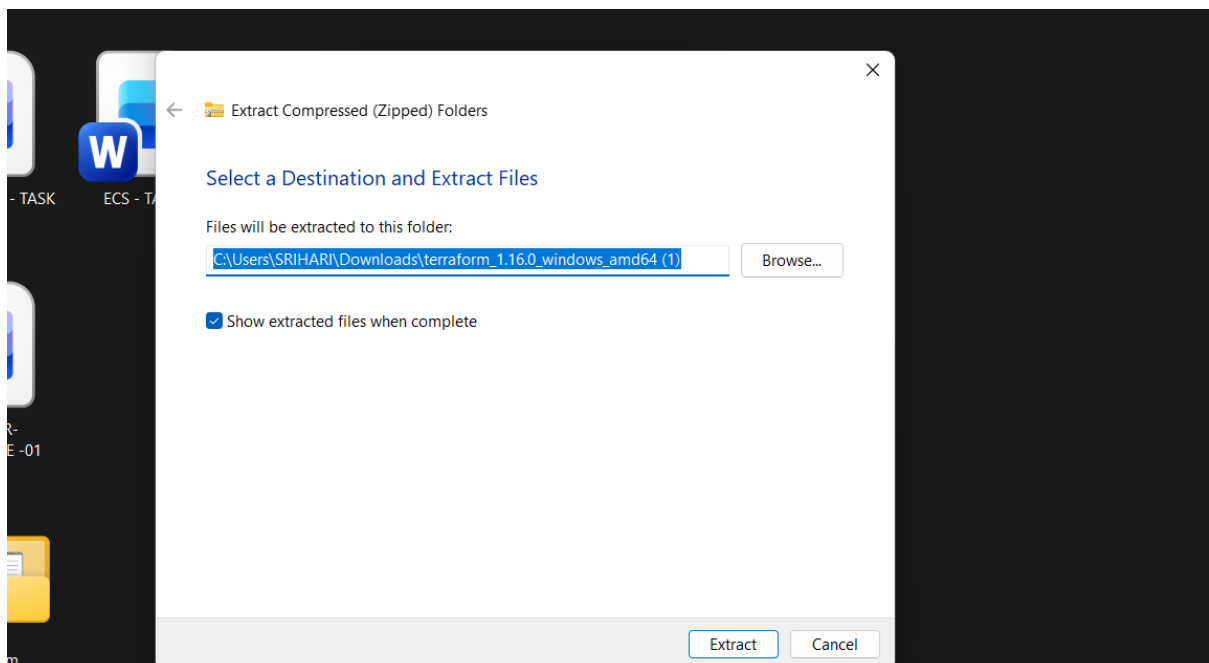
- Open a web browser.
- Go to the official Terraform website: developer.hashicorp.com/terraform/install.
- This page contains Terraform installation options for different operating systems
- From the left menu, select **Windows**.
- Under **Binary download**, choose **AMD64** (for 64-bit Windows systems).
- Click on **Download** to download the Terraform ZIP file.



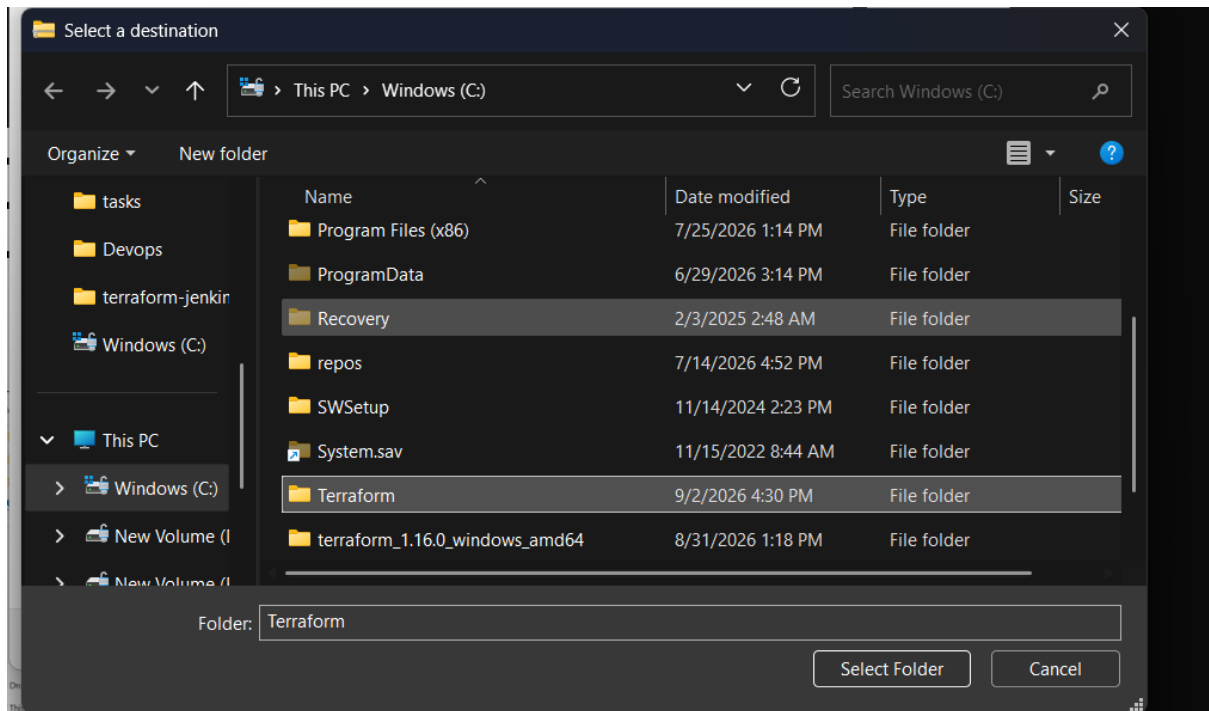
- Open **File Explorer**.
- Navigate to the **Downloads** folder.
- Locate the downloaded file: terraform_1.14.3_windows_amd64.zip.
- Right-click or select **Extract All**.



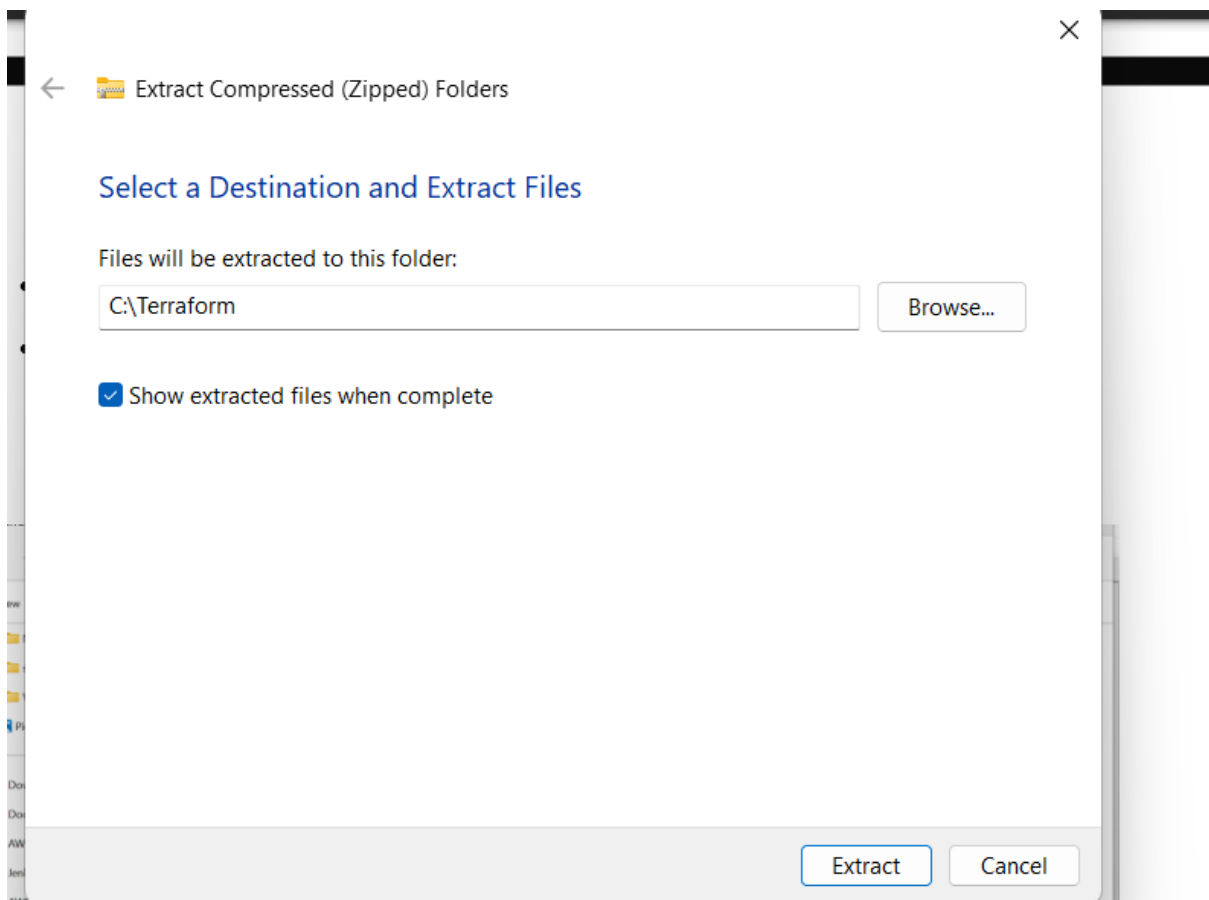
- The “Extract Compressed (Zipped) Folders” window opens.
- Click on **Browse** to choose a custom extraction location.



- Navigate to **This PC → (C:)**.
- Select or create a folder named **Terraform**.
- Click **Select Folder** to confirm the destination.



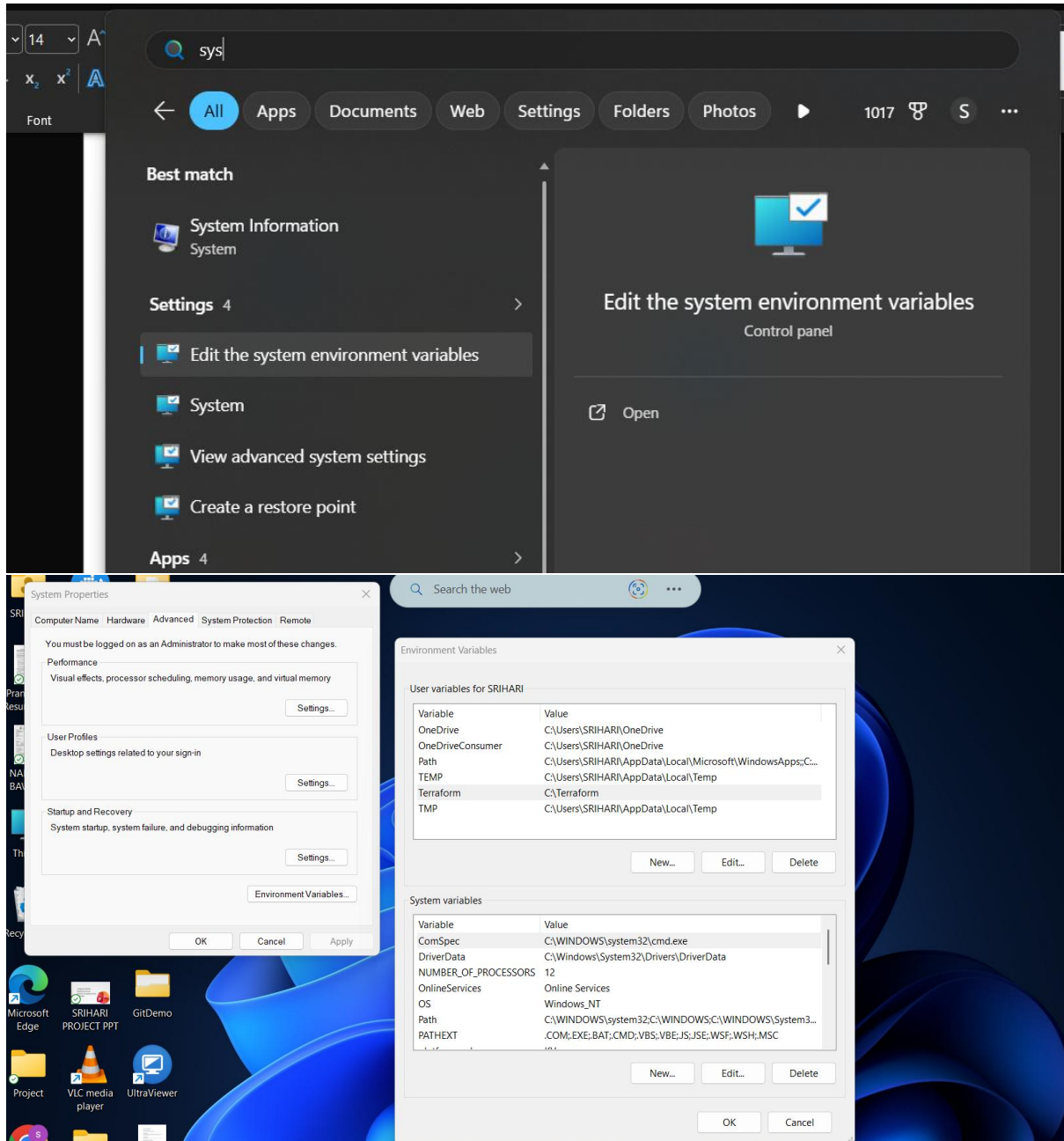
- Confirm that the destination path is C:\Terraform.
- Click on **Extract**.
- Terraform files are extracted to the selected folder.



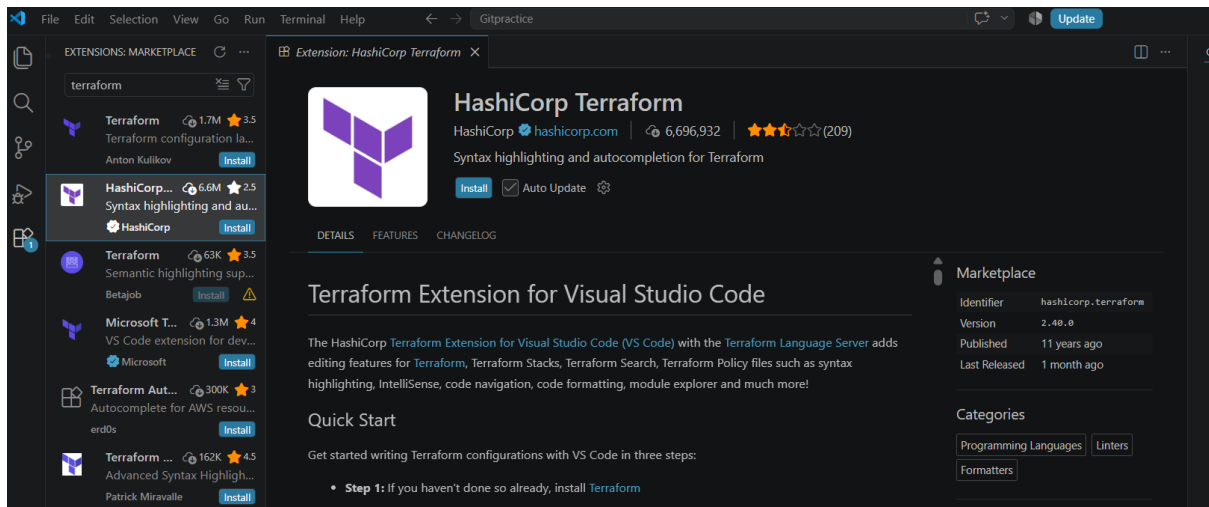
Click on the **Start Menu**.

Search for **Edit the system environment variables**.

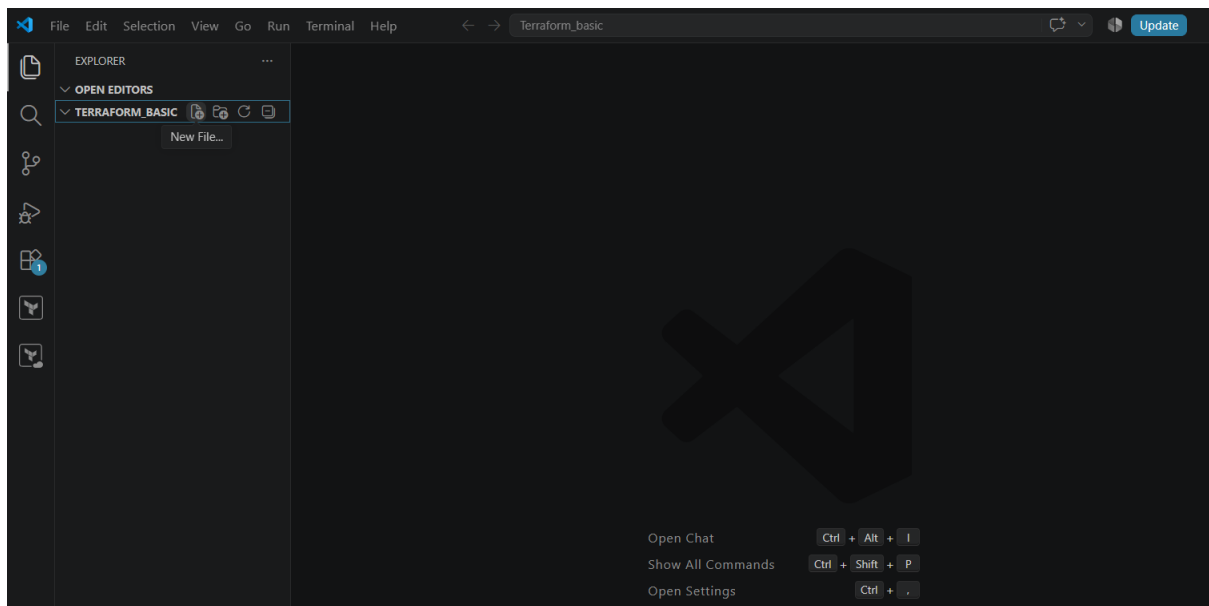
Click **Open** to launch system properties.



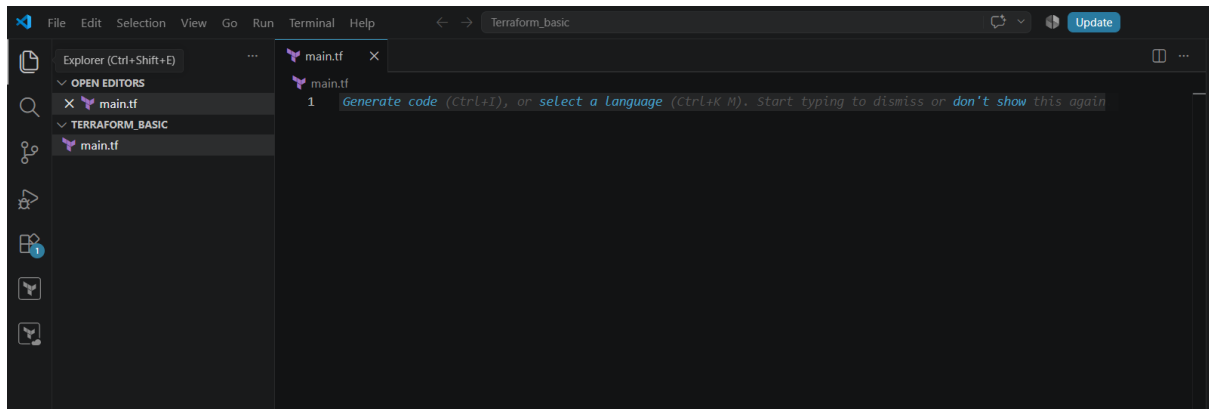
- Open **Visual Studio Code**.
- Go to the **Extensions** tab.
- Search for **Terraform**.
- Install **HashiCorp Terraform** extension (verified publisher).



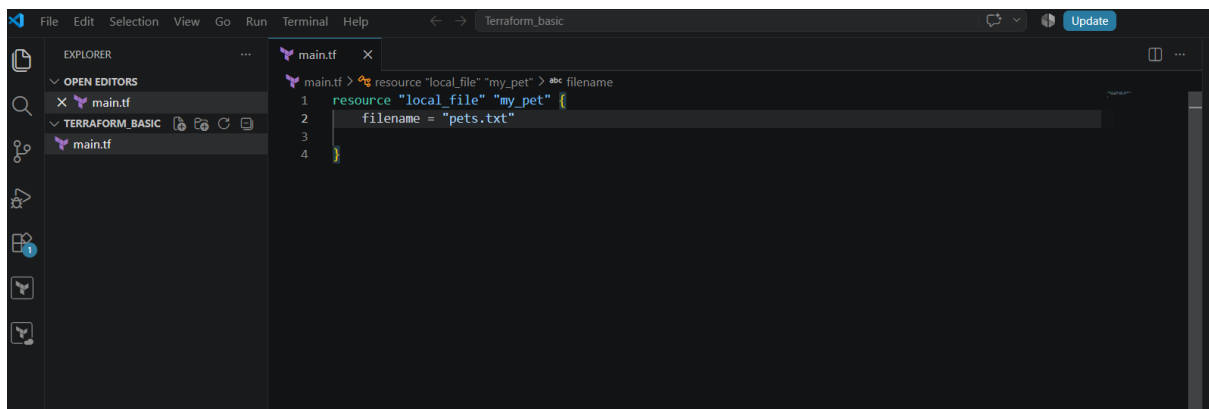
- In VS Code, go to **Explorer**.
- Click on **Open Folder**.
- Select the folder where you want to write Terraform code (example: Terraform Basic).



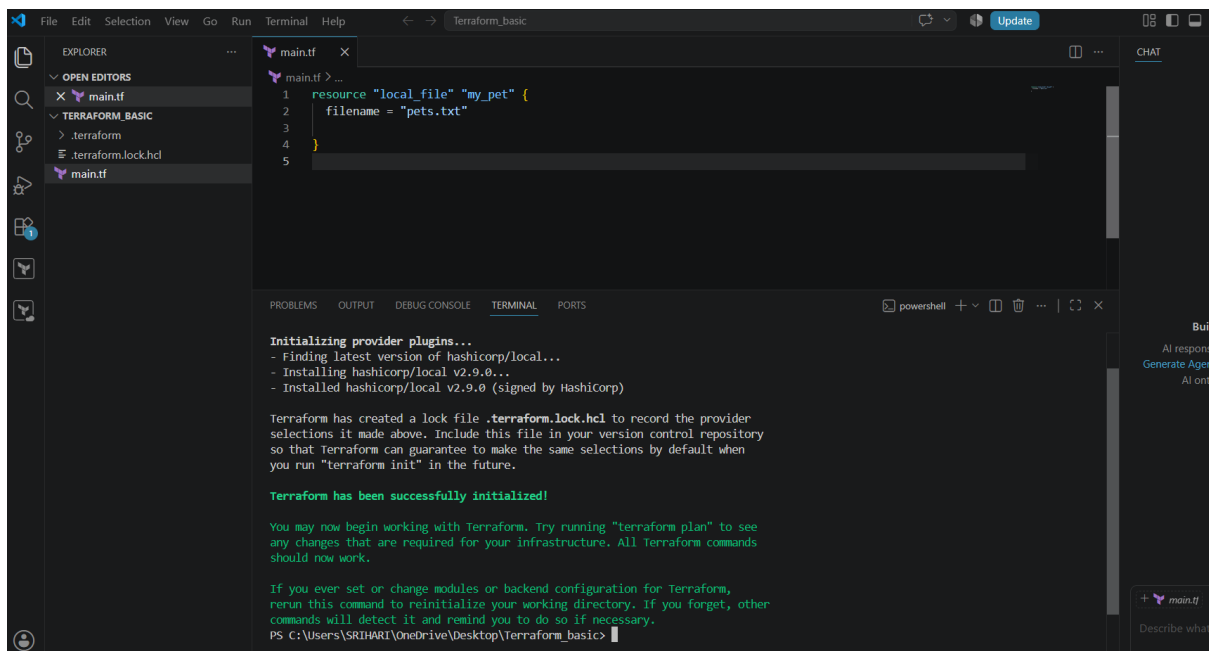
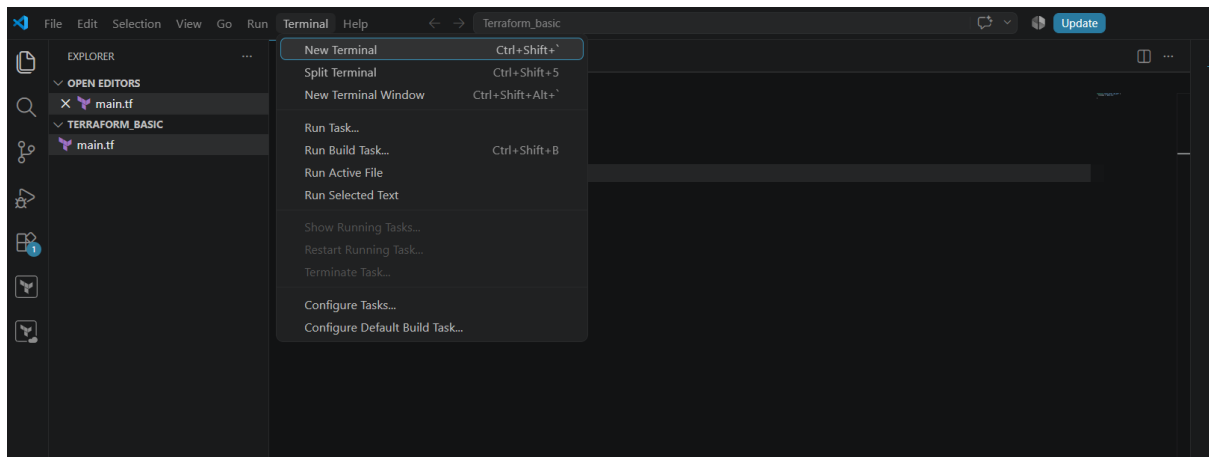
- Inside the opened folder, click **New File**.
- Create a file named **main.tf**.
- This file will be used to write Terraform configuration code.



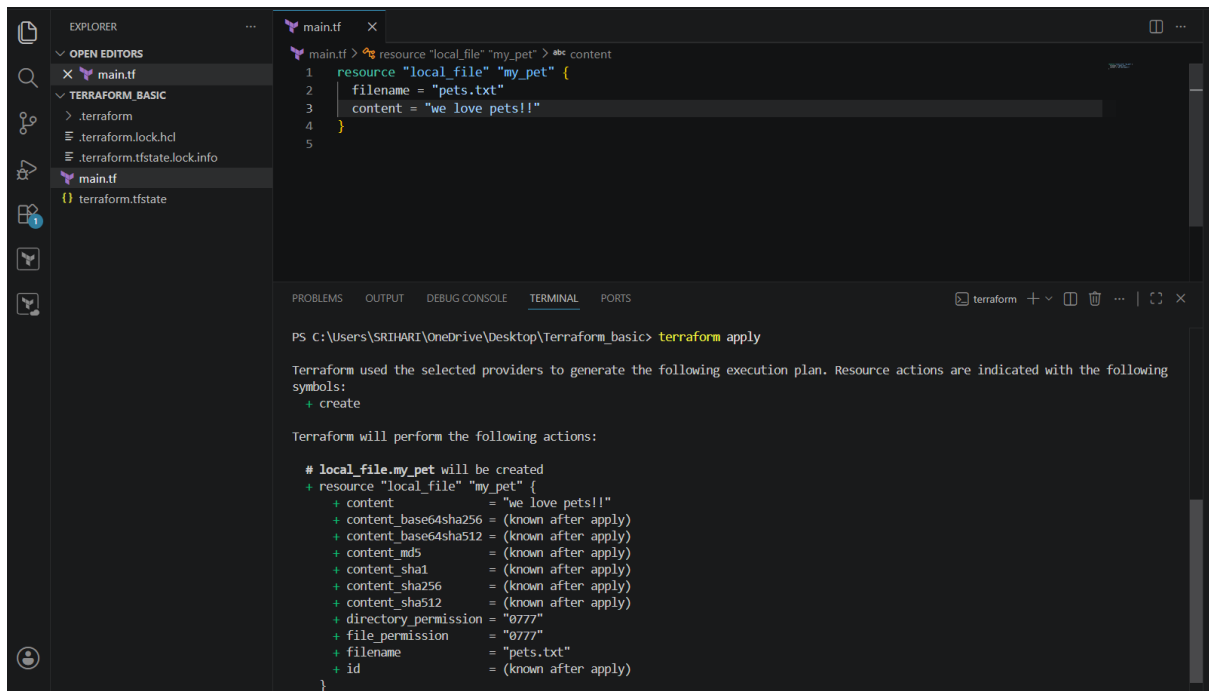
- Open main.tf in VS Code.
- Define a Terraform resource using the **local_file** provider.
- Specify the resource name and the filename.
- Save the file using **Ctrl + S**.



- Open the integrated terminal in VS Code.
- Navigate to the Terraform working directory.
- Run the command:
 - terraform init
- Terraform downloads the required provider plugins.
- A .terraform.lock.hcl file is created.
- Initialization completes successfully.



- Update main.tf to include file content.
- Run the command:
- terraform apply
- Terraform displays the execution plan.
- Type yes to approve the changes.
- Terraform creates the file and updates the state file.



- Replace the local file resource with a **random_pet** resource.
- Define attributes such as prefix, separator, and length.
- Save the updated configuration file.
- Run:
 - `terraform apply`
 - Terraform throws an **inconsistent dependency lock file** error.
 - Error occurs due to provider mismatch after configuration change
 - Fix the issue by running:
 - `terraform init`
 - Terraform reinitializes backend and providers.
 - Required providers are installed successfully.
 - Dependency lock file is updated

```
main.tf | terraform.tfstate
main.tf > resource "random_pet" "my_pet" > * separator
1 resource "random_pet" "my_pet" {
2   prefix = "MR"
3   separator = "."
4   length = "1"
5 }
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell + - - - - -

Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=e60aac62eec3df0c19908f796cff2dcf70dfbec]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/random...
- Reusing previous version of hashicorp/local from the dependency lock file
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by Hashicorp)
- Using previously-installed hashicorp/local v2.9.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
```

Run:

terraform apply

Terraform plans to destroy the old resource and create a new one.

Confirm by typing yes.

Local file resource is destroyed.

Random pet resource is created successfully

```
main.tf | terraform.tfstate
main.tf > resource "random_pet" "my_pet" > * separator
1 resource "random_pet" "my_pet" {
2   prefix = "MR"
3   separator = "."
4   length = "1"
5 }
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell + - - - - -

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply
local_file.my_pet: Refreshing state... [id=e60aac62eec3df0c19908f796cff2dcf70dfbec]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create
- destroy

Terraform will perform the following actions:

# local_file.my_pet will be destroyed
# (because local_file.my_pet is not in configuration)
- resource "local_file" "my_pet" {
  - content = "we love pets!!" -> null
  - content_base64sha256 = "H/Y5b3M7ElFj9ged8wfcWMA1oUJ0vEtfhPrQ9asuf0=" -> null
  - content_base64sha512 = "dM3nbjmfXhjaHK1rzfuJqHhMy52XpPs3L8BbmJ8ZQaq2xYQBWGTamPzBF8CQPa6sC/HnsT6j8vEP5M10NLg==" -> null
  - content_md5 = "99ced8ab4a3d2e8d4f92ac043dabffef" -> null
  - content_sha1 = "e60aac62eec3df0c19908f796cff2dcf70dfbec" -> null
  - content_sha256 = "1ff6396f733b125163f6079d0707c254c29ad68509d2e12d7c73e743d6acb9fd" -> null
  - content_sha512 = "74cde76e399f5e18da1cad6bcdffb89a8784c632e735e93ecdcbf01a1b989f1941aab6c584015864da98fcc117c0903daeaec9
bf1e7b13ea3f2f10fe4dd6d0d2e" -> null
  - directory_permission = "0777" -> null
  - file_permission = "0777" -> null
  - filename = "pets.txt" -> null
  - id = "e60aac62eec3df0c19908f796cff2dcf70dfbec" -> null
}

# random_pet.random_pet.my_pet will be created
+ resource "random_pet" "my_pet" {
  length = 1
  prefix = "MR"
  separator = "."
}
```

```
main.tf
1 resource "random_pet" "my_pet" {
2   prefix = "MR"
3   separator = "."
4   length = 1
5 }
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bfe7b13ea3f2f10fe4dd62d0d2e" -> null
- directory_permission = "0777" -> null
- file_permission = "0777" -> null
- filename = "pets.txt" -> null
- id = "e60aac62eec3df0c199088f796cff2dcf70dfbec" -> null
}

# random_pet.my_pet will be created
+ resource "random_pet" "my_pet" {
+   id = (known after apply)
+   length = 1
+   prefix = "MR"
+   separator = "."
+ }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Destroying... [id=e60aac62eec3df0c199088f796cff2dcf70dfbec]
random_pet.my_pet: Creating...
local_file.my_pet: Destruction complete after 0s
random_pet.my_pet: Creation complete after 0s [id=MR.turtle]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic>
```

Run:

terraform plan

Terraform refreshes the state.

Output shows **no changes**.

Infrastructure matches the configuration

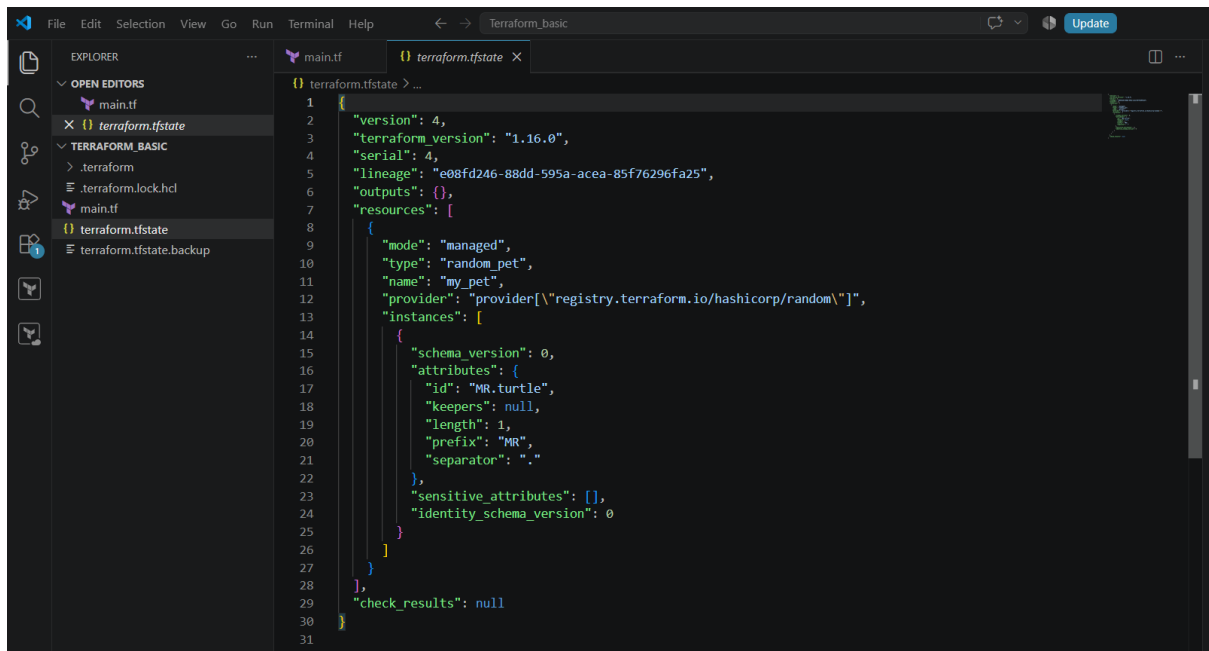
```
random_pet.my_pet: Creation complete after 0s [id=MR.turtle]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform plan
random_pet.my_pet: Refreshing state... [id=MR.turtle]

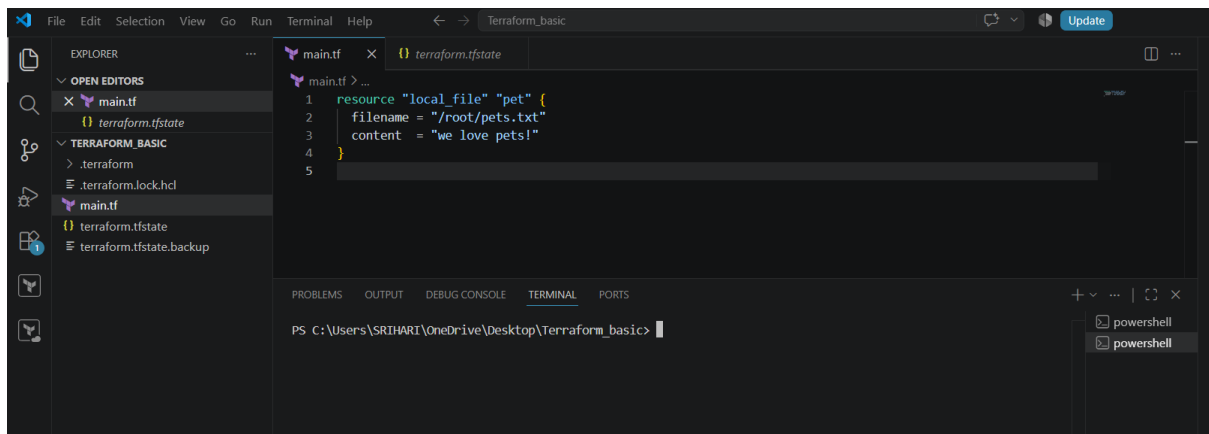
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic>
```

- Open terraform.tfstate in VS Code.
- Observe stored resource details.
- State file contains resource ID, attributes, and provider metadata.
- This file tracks the current infrastructure state.



- Modify main.tf to use **local_file** resource again.
- Define filename and content.
- Save the file.



- Run:
- terraform init
- Terraform reuses existing providers.
- Initialization completes successfully

```
main.tf > ...
1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content  = "we love pets!"
4 }
5
```

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0
- Using previously-installed hashicorp/random v3.9.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
main.tf > ...
1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content  = "we love pets!"
4 }
5
```

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform plan
random_pet.my_pet: Refreshing state... [id=MR.turtle]

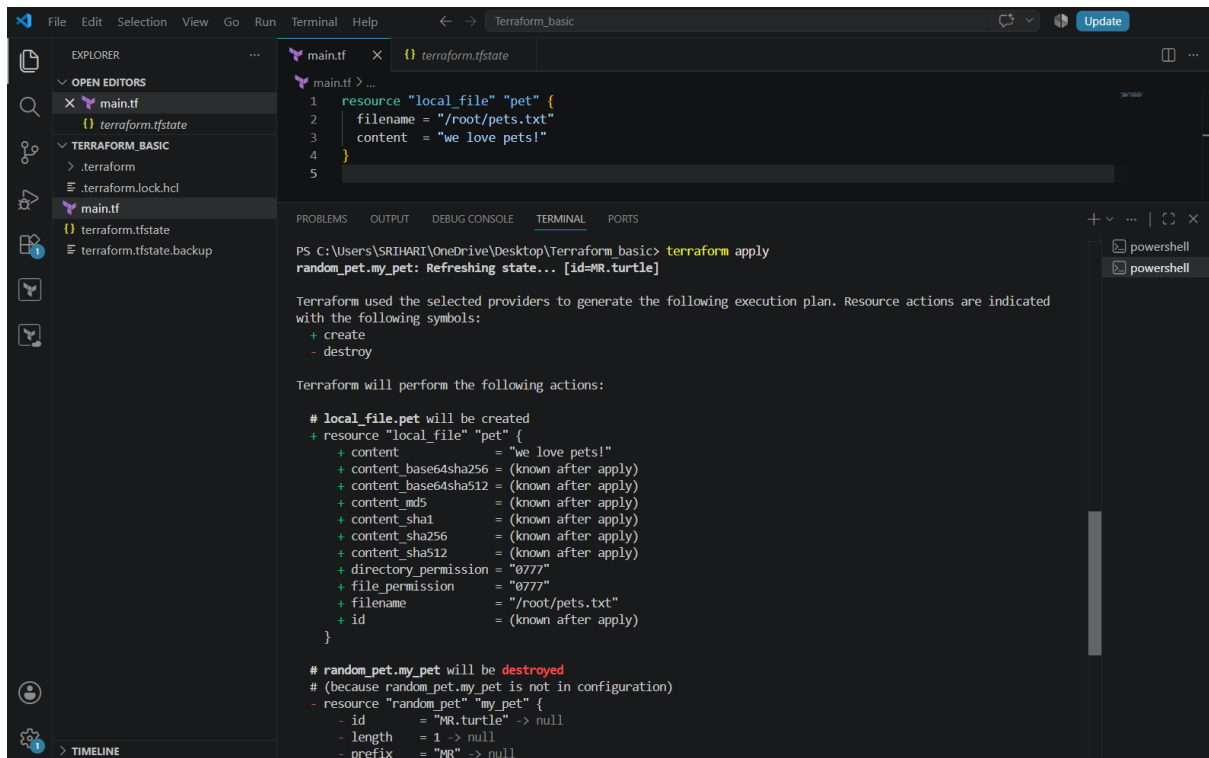
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

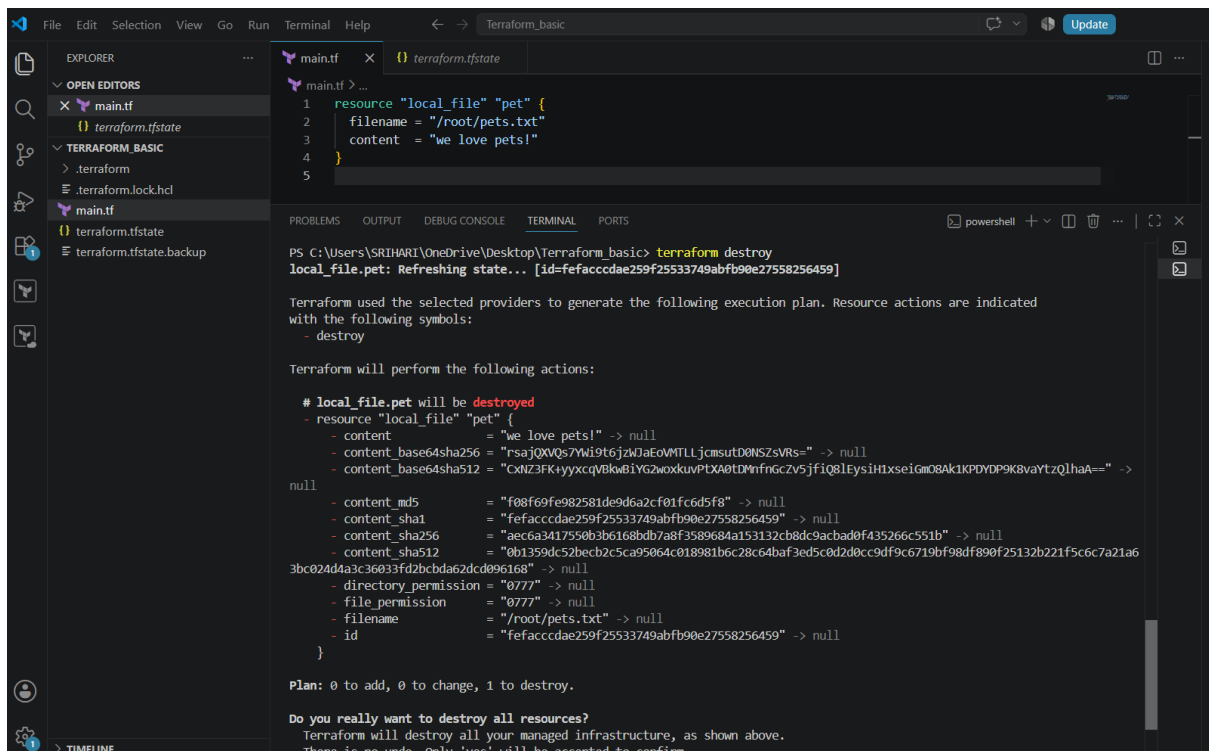
# local_file.pet will be created
+ resource "local_file" "pet" {
  + content          = "we love pets!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "/root/pets.txt"
  + id                = (known after apply)
}

# random_pet.my_pet will be destroyed
```

- Run:
- terraform apply
- Review the execution plan.
- Type yes to confirm.
- Terraform applies changes and updates the state file.



- Open `main.tf` containing the `local_file` resource.
- In the terminal, run:
- `terraform destroy`
- Terraform refreshes the state.
- An execution plan is shown indicating the resource will be destroyed.
- Terraform asks for confirmation before destroying resources



- Modify main.tf.
- Define the resource:
- Resource type: local_file
- Resource name: my_pet
- Filename: pet.txt
- Content: My pet name is Bruno
- Save the file
- Run the command:
- terraform init
- Terraform initializes the backend and providers.
- Then run:
- terraform plan
- Terraform shows that a **new local_file resource will be created**

```

1 resource "local_file" "my_pet" {
2   filename = "pet.txt"
3   content  = "my pet name is browine"
4 }
5

```

```

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform plan

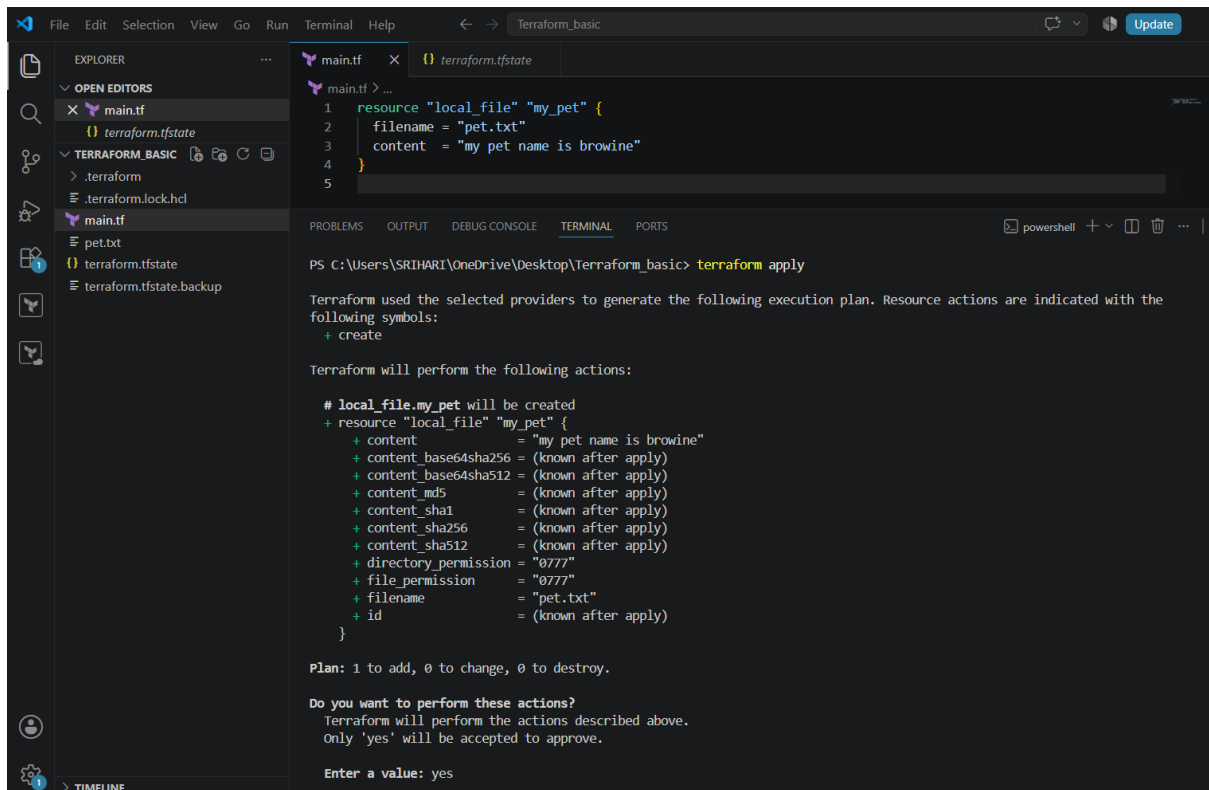
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

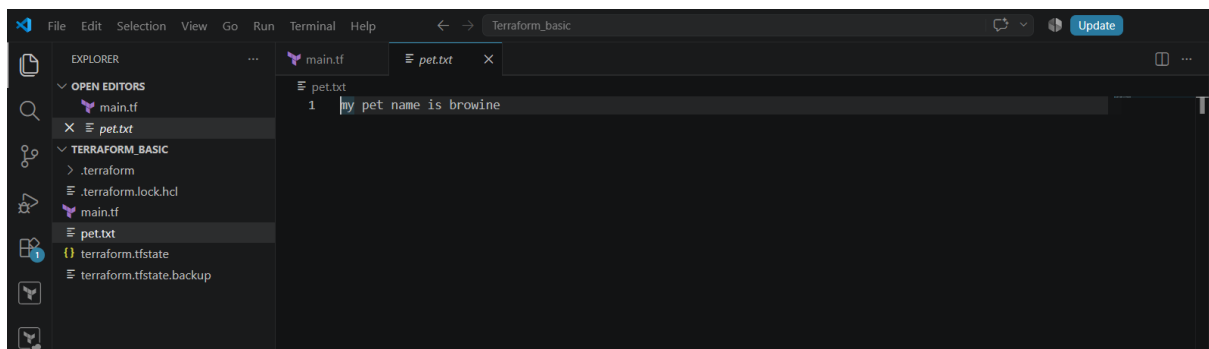
# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+   content  = "my pet name is browine"

```

- Run:
- terraform apply
- Terraform displays the execution plan.
- Type yes to approve the changes.
- Terraform creates the pet.txt file.
- Apply completes successfully.

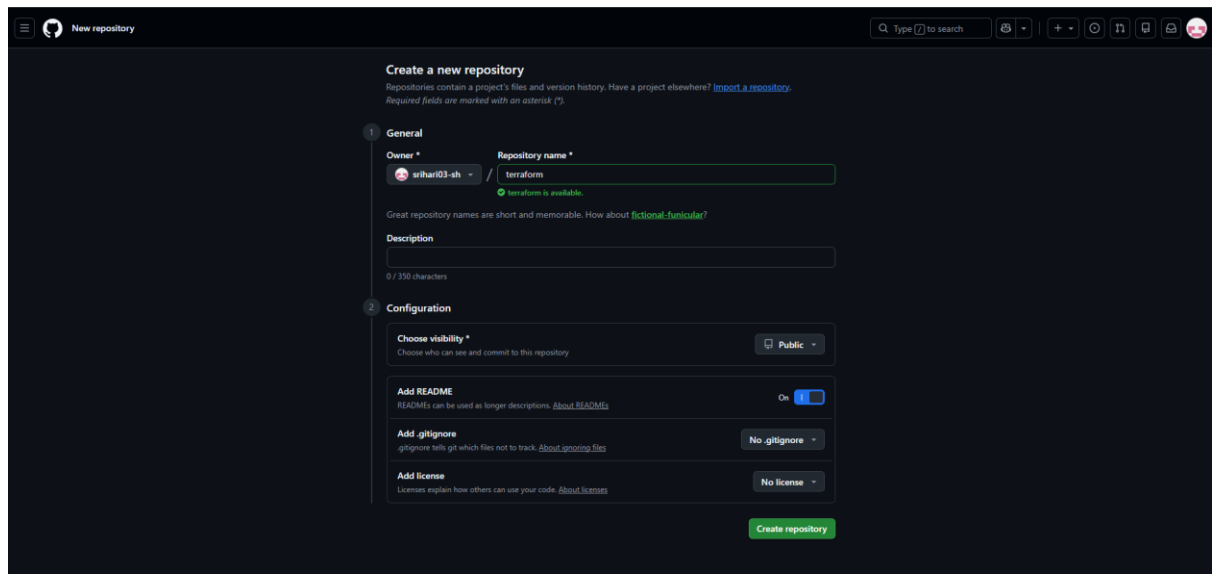


- In VS Code Explorer, locate the newly created pet.txt file.
- Open pet.txt.
- Verify that the content is:
- My pet name is Browine
- This confirms Terraform created the file successfully.



4. Integrate a sample Terraform template in Jenkins.

- Open GitHub and click **New repository**.
- Enter the repository name as **terraform**.
- Choose visibility as **Public**.
- Enable **Add README** option.
- Click **Create repository** to finalize



The screenshot shows the GitHub 'Create a new repository' page. The 'General' section is active, showing the repository name 'terraform' and the owner 'srinani03-sh'. The 'Configuration' section is also visible, showing options for visibility (Public), adding a README, .gitignore, and a license. The 'Create repository' button is at the bottom right.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * srinani03-sh / Repository name * terraform
terraform is available.

Great repository names are short and memorable. How about [fictional-funicular](#)?

Description
0 / 350 characters

2 Configuration

Choose visibility * Public
Choose who can see and commit to this repository

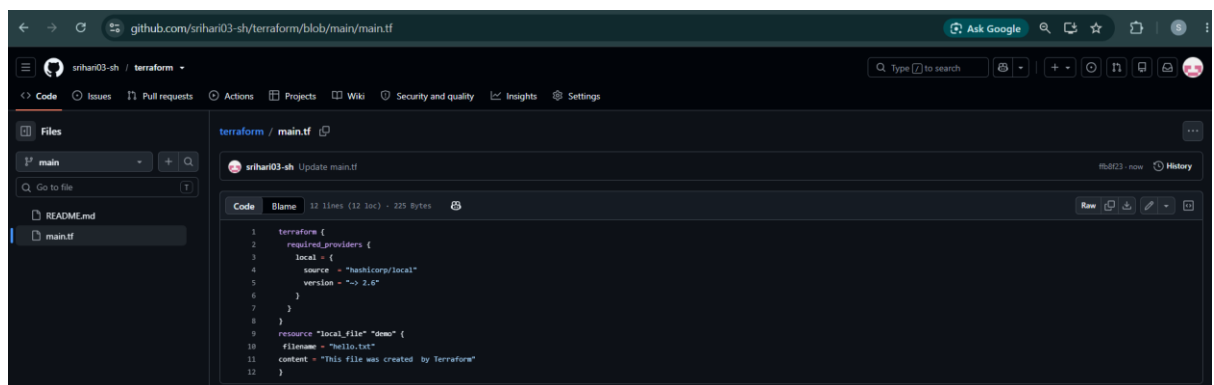
Add README On
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

Create repository

- Open the newly created repository.
- Create or edit /
 - Add Terraform configuration:
- Define required providers.
- Use local_file resource.
- Add file name and content.
- Commit the changes to the main branch



The screenshot shows the GitHub repository page for 'terraform' by 'srinani03-sh'. The 'main' branch is selected, and the file 'main.tf' is open. The code content is as follows:

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "~> 2.6"
6     }
7   }
8 }
9 resource "local_file" "demo" {
10   filename = "hello.txt"
11   content = "This file was created by Terraform"
12 }
```

- Log in to the Jenkins EC2 instance.
- Add HashiCorp repository.
- Install Terraform using package manager.
- Terraform binary is downloaded and installed successfully.
- Verify installation completes without errors.

Commands:

```
sudo yum install -y yum-utils
```

```
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

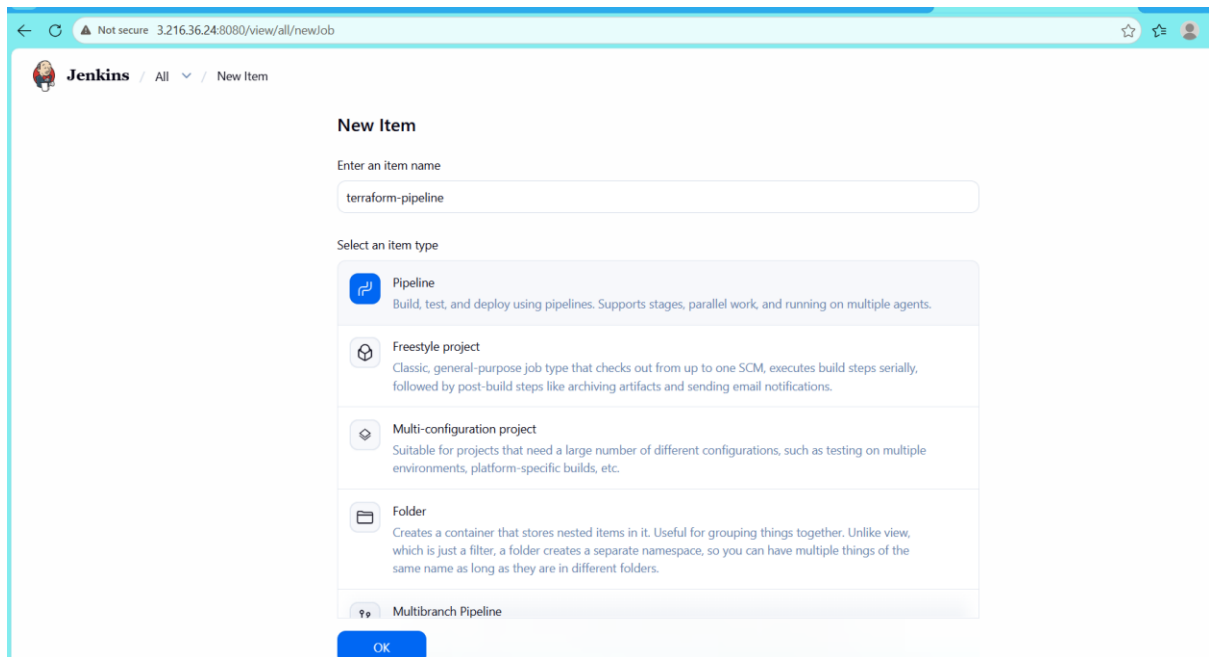
```
sudo yum install -y terraform
```

```
[root@ip-172-31-6-115 ~]# sudo yum install -y yum-utils
Last metadata expiration check: 0:06:36 ago on Wed Sep 2 13:54:40 2026.
Package dnf-utils-4.3.0-13.amzn2023.0.6.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-6-115 ~]# sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[root@ip-172-31-6-115 ~]# sudo yum install -y terraform
Hashicorp Stable - x86_64                                42 MB/s | 1.8 MB  00:00
Last metadata expiration check: 0:00:01 ago on Wed Sep 2 14:01:55 2026.
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
terraform x86_64 1.16.0-1 hashicorp 34 M
Transaction Summary
Install 1 Package
Total download size: 34 M
Installed size: 115 M
Downloading Packages:
terraform-1.16.0-1.x86_64.rpm                                171 MB/s | 34 MB  00:00
Total
Hashicorp Stable - x86_64
Importing GPG key 0xA621E701:
  Userid : "HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>"
  Fingerprint: 798A EC65 4B5C 1542 8C8E 42EE AA16 FC8C A621 E701
  From : https://rpm.releases.hashicorp.com/gpg
Key imported successfully
i-00ca6f09e98a51fdb (Jenkins-Server)
PublicIPs: 3.216.36.24 PrivateIPs: 172.31.6.115
```

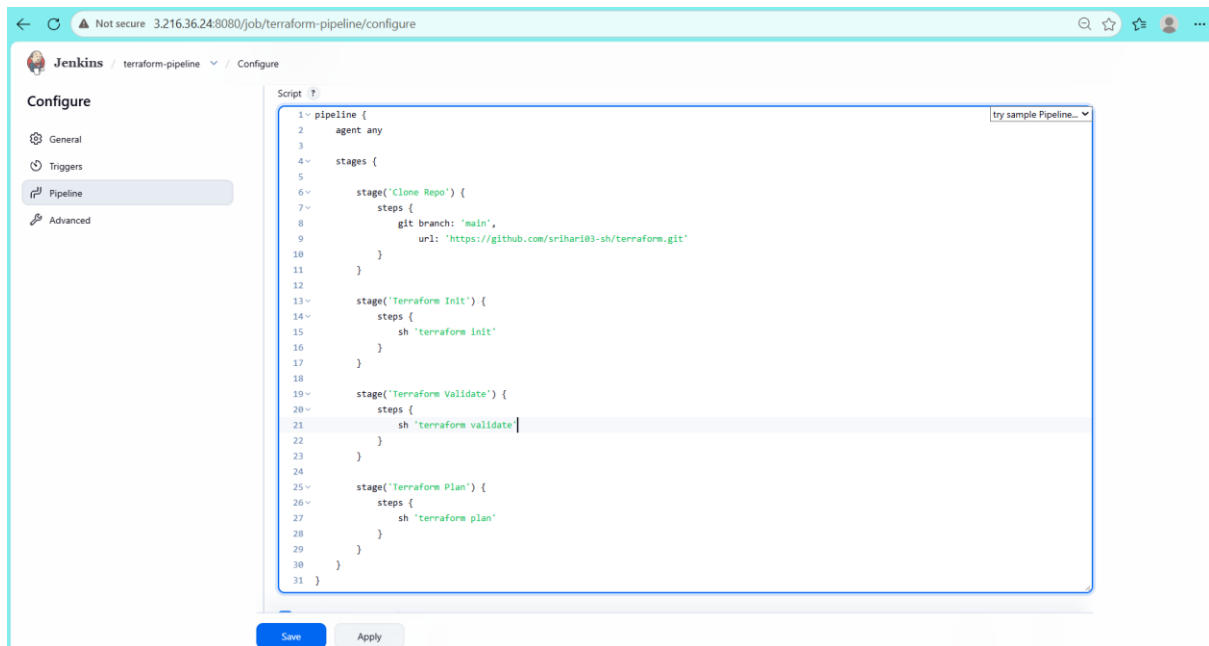
- Run:
- terraform -v
- Terraform version is displayed.
- Confirms Terraform is successfully installed on Jenkins server.

```
[root@ip-172-31-6-115 ~]# terraform -v
Terraform v1.16.0
on linux amd64
[root@ip-172-31-6-115 ~]#
i-00ca6f09e98a51fdb (Jenkins-Server)
PublicIPs: 3.216.36.24 PrivateIPs: 172.31.6.115
```

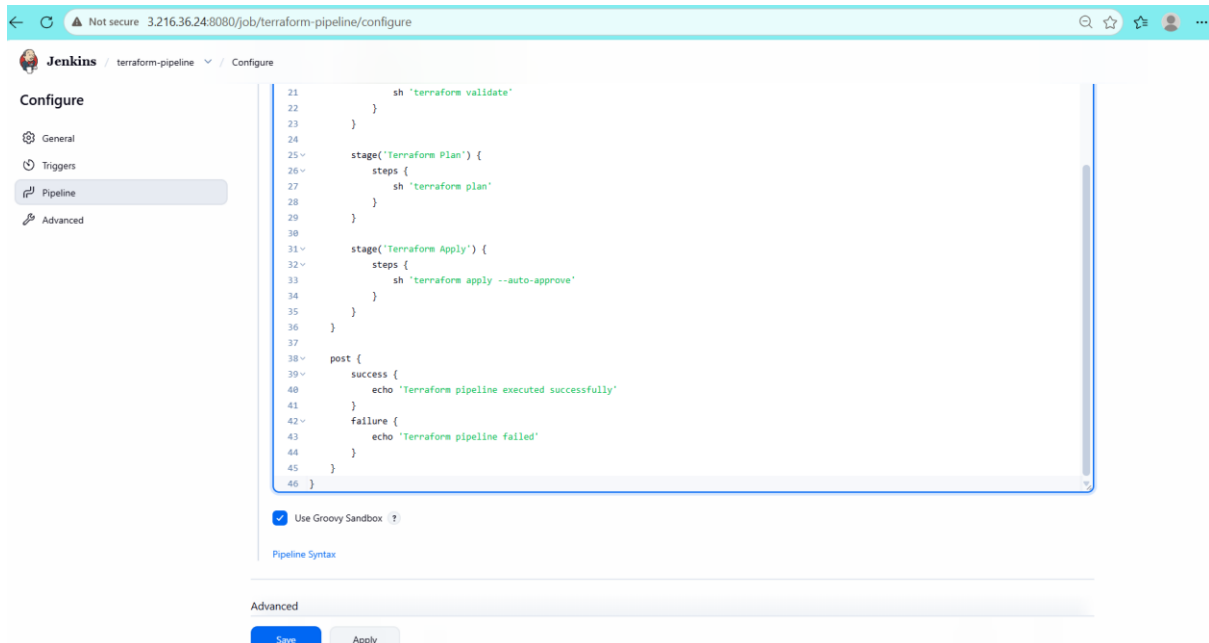
- Open Jenkins dashboard.
- Click **New Item**.
- Enter job name: terraform-pipeline.
- Select **Pipeline** as job type.
- Click **OK**.



- In Pipeline configuration, select **Pipeline script**.
- Add stages:
 - Clone GitHub repository
 - Terraform Init
 - Terraform Validate
 - Terraform Plan
- Use GitHub repository URL in clone stage.



- Add **Terraform Apply** stage with auto-approve option.
- Add **post actions**:
- Success message on pipeline success.
- Failure message on pipeline failure.
- Save the pipeline configuration



- Trigger the pipeline build.
- Jenkins executes stages sequentially:
- Clone Repo
- Terraform Init

- Terraform Validate
- Terraform Plan
- Terraform Apply
- All stages complete successfully (green status)



- Log in to Jenkins server.
- Navigate to workspace directory:
- /var/lib/jenkins/workspace/terraform-pipeline
- List files using ls.
- Terraform files, state file, and output files are present.

```

https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-00ca6f09e98a51fdb&osUser=ec2-user&region=us-...

AWS IAM VPC EC2 S3 Route 53 Certificate Manager

A newer release of "Amazon Linux" is available.
Version 2023.12.20260817:
Version 2023.12.20260831:
Run "/usr/bin/dnf check-release-update" for full release and version update info

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Wed Sep 2 14:11:30 2026 from 18.206.107.27
[ec2-user@ip-172-31-6-115 ~]$ sudo su -
Last login: Wed Sep 2 14:11:41 UTC 2026 on pts/1
[root@ip-172-31-6-115 ~]# cd /opt/
[root@ip-172-31-6-115 opt]# cd /var/lib/jenkins/workspace/terraform-pipeline
[root@ip-172-31-6-115 terraform-pipeline]# ls
README.md hello.txt main.tf terraform.tfstate
[root@ip-172-31-6-115 terraform-pipeline]#

```

i-00ca6f09e98a51fdb (Jenkins-Server)
PublicIPs: 3.216.36.24 PrivateIPs: 172.31.6.115